CASE-STUDY PRACTICE | CAPSTONE 2 | STUDY EDITION

Ad CTR Prediction: Online Learning at Scale

Scenario questions on estimating the probability that an ad is clicked, with online learning, sparse models and calibration, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on data, a model or a serving fleet: figures are the paper's or arithmetic from stated inputs, and traffic volumes, coefficient counts, key sizes and replica counts are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 6

Parts at a glance

1 Framing the problem: probabilities, labels and the sizing 7

2 Features and data: hashing, rare features and subsampling 11

3 The learning algorithm: FTRL-Proximal and per-coordinate rates 16

4 Serving, evaluation and operations 22

Sources and confidence 27

Every question, by part

Part 1 Framing the problem: probabilities, labels and the sizing 7

01 Why a probability, and not a yes or no? Framing 7

02 When does a click count? Labels 8

03 Size the prediction traffic Estimation 9

04 Does 16-bit storage save 75%? Sizing 10

Part 2 Features and data: hashing, rare features and subsampling 11

05 Hash the features, or keep them exact? Design choice 11

06 Which rare features deserve a slot? Memory 12

07 Subsample the non-clicked queries Data 13

08 Review the sampler Code review 14

Part 3 The learning algorithm: FTRL-Proximal and per-coordinate rates 16

09 Why not plain gradient descent with an L1 penalty? Algorithm choice 16

10 Why does each coordinate get its own learning rate? Algorithm 17

11 Trace one coordinate by hand Calculation 18

12 Review the FTRL update Code review 19

13 Counts instead of squared gradients, and many variants at once Memory 20

Part 4 Serving, evaluation and operations 22

14 Replicate a model that never stops changing Serving 22

15 How good is a metric, and how noisy? Evaluation 23

16 Calibrate, and say how sure the model is Evaluation 24

17 Ten times the traffic Scale 25

18 Fake clicks and a broken signal Risk 26

The case in one page

A summary of the case study to read before the questions.

Problem	For every ad matched to a search query, estimate the probability that it will be clicked. An auction combines that probability with the advertiser's bid.
Core idea	Online logistic regression trained with FTRL-Proximal. Each coordinate has its own learning rate, and an L1 penalty drives unneeded coefficients to exactly zero so the served model is small.
Metric	AucLoss (1 minus AUC), LogLoss and SquaredError, reported as relative changes against a baseline on the same data, scored by progressive validation. A calibration layer corrects bias.
Headline results	Per-coordinate learning rates cut AucLoss by 11.2% against one global rate. FOBOS needs 38% more non-zero coefficients than FTRL-Proximal at equal accuracy (source-confirmed).
Memory tricks	16-bit fixed point saves 75% of the RAM for coefficient values. Bloom filter inclusion saves 66% of training RAM for 0.008% AucLoss (source-confirmed).
Surprise	The paper tried feature hashing and rejected it: no projection below several billion features without observable loss (source-confirmed).
Scale	Billions of predictions a day, billions of dimensions, hundreds of non-zero values per example (source-confirmed). Illustrative: 10 billion a day, 2 billion non-zero coefficients, 20 GB per replica.
Easy-to-miss traps	The 75% saving covers values, not keys: 37.5% for the whole table. A 2% click rate per ad is 5.9% per query for a query-level sample. A constant model has 98% accuracy and chance AUC. A weight used after the state changed gives a quiet 0.5% error.

Numbers to know

Each figure carries the tag that says how it was obtained. Traffic, feature counts, key sizes and replica counts are illustrative, because the paper gives scale only in qualitative terms.

Quantity	Value	Evidence
Scale statements	Billions of events a day; billions of dimensions; hundreds of non-zero values per instance; billions of coefficients; half the unique features occur once in some models	source-confirmed (the paper)
Average and peak rate	10,000,000,000 ÷ 86,400 = 115,741/s; at 3 times, 347,222/s	illustrative; recomputed
Peak lookups	347,222 × 300 = 104 million a second; about 10 cores at 10 million lookups a second per core	illustrative; recomputed
q2.13 format	1 sign bit + 2 integer bits + 13 fraction bits = 16 bits; step 2 to the power minus 13; 75% less RAM for coefficient values than 64-bit floats	source-confirmed (the paper)
Whole table with keys	2 billion coefficients: 8-byte key + 8-byte float = 32 GB; 8-byte key + 2-byte value = 20 GB; saving 37.5%	illustrative; recomputed
Fleet memory	80 copies: 2.56 TB with floats, 1.60 TB with q2.13 values; 0.96 TB saved	illustrative; recomputed
Hashing break-even	Hashed dense array B × 2 bytes against keyed table N × 10 bytes: hashing smaller only if B below 5N; B = 2 to the power 32 gives 8.6 GB	illustrative; recomputed
Hashing result	Unable to project below several billion features without observable loss	source-confirmed (the paper)
Probabilistic inclusion (Table 2)	Bloom n=2 66% RAM, 0.008%; Bloom n=1 55%, 0.003%; Poisson p=0.03 60%, 0.020%; Poisson p=0.1 40%, 0.006%	source-confirmed (the paper)
Query-level sample	1 − 0.98^3 = 5.88% of queries clicked; r = 0.10 keeps 15,292,720 of 100,000,000 (6.5 times fewer); weighted impressions 300,000,000	illustrative; recomputed
Without weights	6,000,000 clicks ÷ 45,878,160 impressions = 13.1% against 2%; per-event dropping gives 16.9%	illustrative; recomputed
Table 1 (sparsity)	RDA +3% non-zeros, 0.6% AucLoss; FOBOS +38%, 0.0%; OGD-Count +216%, 0.0%; RDA and FOBOS on a smaller dataset, OGD-Count on a full-scale one	source-confirmed (the paper)
Per-coordinate rates	AucLoss reduced 11.2% against a global rate; reductions of 1% considered large; coin rates 0.031 against 0.316	source-confirmed (the paper); coin numbers recomputed
Hand trace	alpha 0.1, beta 1, L1 0.5: after two clicks z = −1.0, n = 0.5, weight 0.0293; after one click weight 0	illustrative; recomputed
Update-order fault	Correct weight after four clicks 0.0802; with the weight read after n changes 0.0798	illustrative; recomputed
Count approximation	P × N ÷ (N + P) = 9.9 for P = 10, N = 990; sharing across 10 variants: 140 to 32 bytes per coordinate	source-confirmed (formula); recomputed
Snapshot push	80 × 20 GB = 1.6 TB; every 5 minutes is about 5.3 GB a second	illustrative; recomputed
Metric noise	417 million events an hour: 95% margin on a 2% rate 0.07%; a 0.1% slice: 2.1%	illustrative; recomputed
Isotonic pooling	Observed 0.020, 0.046, 0.044, 0.150 become 0.020, 0.045, 0.045, 0.150	illustrative; recomputed

Quantity	Value	Evidence
Ten times the traffic	1,157,407 a second average, 3,472,222 peak; 15.3 billion training events a day; noise down 3.2 times	illustrative; recomputed
Fake clicks	10 million impressions: 200,000 clicks plus 1,000,000 fake = 12%, six times the true 2%	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Feature identity	Exact keys in a table, no hashing	A hashed dense array only if B is below 5N and the accuracy cost is tolerable	source-confirmed; reasoned
Rare-feature control	Bloom filter inclusion	Poisson inclusion if running filters is too costly	source-confirmed
Training algorithm	FTRL-Proximal with per-coordinate rates	Plain OGD if sparsity at serving does not matter	source-confirmed; reasoned
Update style	Online, one example at a time	Batch retraining if freshness is not needed and reproducibility is	source-confirmed; reasoned
Negative sampling	Per query, weights 1 and 1 over r	No sampling if training cost is no concern	source-confirmed
Validation	Progressive validation, hourly, relative changes	A held-out set when each model must be scored on frozen data	source-confirmed; reasoned
Calibration	Isotonic regression	A power law if a simple, closed form is needed	source-confirmed
Propagation to replicas	Decided by the measured size of a delta	Snapshots if the delta approaches the model size	reasoned

PART 1

Framing the problem: probabilities, labels and the sizing

Why the auction needs a calibrated probability, how late clicks bend the labels, and what a prediction costs in compute and in memory.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Click-through rate (CTR)	The share of ad impressions that are clicked. A CTR of 2% means 2 clicks per 100 impressions. Here it is estimated per impression, as the probability P(click | q, a) that ad a is clicked when shown for query q.
Calibration	A set of predictions is calibrated if, among the impressions where the model said p, the observed click rate is close to p. It is a property of a group of predictions, not of one.
Accuracy, and the class-imbalance trap	Accuracy is the share of predictions that match the label. When 98% of impressions are not clicked, a model that always says no click scores 98%, and it knows nothing.
AUC and AucLoss	AUC is the chance that a random clicked impression gets a higher predicted probability than a random unclicked one: 1.0 is perfect and 0.5 is chance. The paper reports AucLoss, 1 minus AUC, so smaller is better.
Impression, click and join window	An impression is one ad shown. Its click, if any, arrives later and on another stream. The join window is how long the system waits for a click before it labels the impression 0.
Sparse feature vector	A list with billions of possible positions, almost all zero. Each impression has only hundreds of non-zero entries, such as one for each query word present.
Quantization (q2.13)	Storing a number on a coarse grid. The paper's q2.13 uses 16 bits: a sign bit, 2 bits before the binary point and 13 after, so values step by 2 to the power of minus 13.
Replica	A full copy of the served model in one location. The paper replicates the model to many data centers, so memory is paid once for each copy.
Where to read	McMahan et al.: Ad Click Prediction: a View from the Trenches (KDD 2013)

01 Why a probability, and not a yes or no?

FRAMING

SCENARIO

A teammate proposes treating ad click prediction as a classifier that outputs click or no click, judged by accuracy. The paper says that an auction takes the advertisers' bids and an estimate of P(click | q, a) as inputs. Argue for or against the teammate's design.

HOW TO THINK ABOUT IT

Ask what the consumer of the output needs, then test a trivial model against the proposed metric.

Q What does the auction need from the model?

A A number it can combine with a bid. The paper names both the bids and the estimate P(click | q, a) as inputs to the auction (source-confirmed).

- For a per-click bid, one ad's expected value is bid times probability. The paper does not give the formula, so treat that as reasoning, not as a quotation.

Q Why is a yes or no not enough?

A Ad A bids 2.00 with a 1% click chance, so its expected value is 0.020 per impression. Ad B bids 0.50 with a 5% chance, so 0.025. B should rank first.

- A classifier that says no click to both has no basis to order them. One that says yes to both ignores the 5 times difference. The size of the probability decides the order.

Q What does accuracy report for a model that knows nothing?

A At a 2% click rate (the paper's own example figure), always predicting no click is right on 98% of impressions.

- It gives every ad the same probability, so its AUC is 0.5, chance level, and its AucLoss is 0.5. Accuracy cannot tell it from a good model.

Q What does the paper use instead?

A AucLoss, LogLoss and SquaredError, reported as relative changes against a baseline on the same data (source-confirmed).

- The stated reason: even for perfect predictions, absolute LogLoss depends on the click rate. At 50% the best possible LogLoss is much higher than at 2%.

Q Reframe it.

A Start from what the downstream consumer does with the number. Here the consumer multiplies and ranks, so the probability must be right in size as well as in order.

Basis: The auction inputs and the metric choice are from the paper (source-confirmed); the two-ad example and the 98% figure are illustrative arithmetic; AUC 0.5 for a constant model is standard.

02 When does a click count?

LABELS

SCENARIO

Impressions and clicks come from two streams. The paper says its data is provided by a streaming service based on Photon and says nothing about how long it waits for a click. You must choose a join window. A colleague says: wait a long time, so no label is ever wrong.

HOW TO THINK ABOUT IT

Price the cost of waiting as well as the cost of being early. Then ask whether the error is the same everywhere.

Q What does the paper actually say about the join?

A Only that the data comes from a streaming service based on Photon, a system for joining continuous data streams (source-confirmed).

- The window length, and what happens to a click after the cutoff, are not in the paper. Any figure for them is your own design.

Q What are the two errors?

A Too short: a click that arrives after the window is already stored as a 0, so a real click is labeled as a non-click.

- Too long: every impression waits out the window before it can be trained on, so the model learns from older data.

Q How big can the bias be?

A If 1% of true clicks arrive after the window (illustrative), observed CTR is 1% lower in relative terms: 1.98% instead of 2.00%.

- A calibration layer can fix a uniform bias. If late clicks are 0.5% of clicks in one market and 3% in another, the observed rates are 1.99% and 1.94%, and one correction is wrong for both.

Q How would you choose the window?

A Measure the delay distribution of clicks and pick the point where the share missed is small and the added staleness is acceptable.

- Track missed-click share by slice, because it differs by market and ad format. Consider correcting late clicks rather than ignoring them.

Q Reframe it.

A Waiting is not free. Name both costs of a timing choice, then check whether the error is uniform, because a uniform error can be calibrated away and a skewed one cannot.

Basis: The paper's statement about Photon is source-confirmed; the 1% late share, the two markets and the resulting rates are illustrative arithmetic; the selection method is reasoned.

03 Size the prediction traffic

ESTIMATION

SCENARIO

The paper says only that a typical model predicts on billions of events a day and that its own system predicts many billions of times a day. Plan for 10 billion predictions a day with 300 non-zero features each. Is the pressure on compute or on memory?

HOW TO THINK ABOUT IT

Convert to per-second figures, count the table lookups, then compare with what one core can do before looking at memory.

Q What are the rates?

A 10,000,000,000 ÷ 86,400 = 115,741 predictions a second on average. At an assumed peak of 3 times, 347,222 a second.

- The 10 billion and the ratio of 3 are illustrative. A ratio of 3 is lower than a one-region consumer app because search spans many time zones.

Q How many queries is that?

A Each query scores several candidate ads. With an assumed 20 candidates, 347,222 ÷ 20 = about 17,361 queries a second at peak.

- The paper does not give candidates per query. Say which number you used.

Q How much compute is a prediction?

A A sparse dot product over about 300 values, then a sigmoid. At peak that is 347,222 × 300 = about 104 million table lookups a second.

- At an assumed 10 million lookups a second per core, about 10 cores in total, spread over the fleet. Compute is small.

Q Then what is large?

A Memory. Every replica holds the whole model, and the model has billions of coefficients (source-confirmed).

- Question 4 sizes one copy. The fleet pays for it many times over.

Q Reframe it.

A Convert each requirement into the resource it presses on. A huge request count can still be cheap per request, and the real limit sits somewhere else.

Basis: The qualitative scale is from the paper (source-confirmed); the 10 billion, ratio of 3, candidates per query and lookups per core are illustrative; arithmetic recomputed.

04 Does 16-bit storage save 75%?

SIZING

SCENARIO

The paper says that storing coefficients as 16-bit fixed point instead of 64-bit floats saves 75% of the RAM for coefficient storage with no measurable loss. A manager reads that and plans the serving fleet's memory as one quarter of the float size. The model has 2 billion non-zero coefficients.

HOW TO THINK ABOUT IT

Ask what is stored beside each coefficient value. The saving applies to what the paper measured, which may not be everything in the table.

Q What does the paper's figure cover?

A Coefficient storage: 64 bits down to 16 bits, a factor of 4, which is 75% (source-confirmed). For 2 billion coefficients, 16 GB becomes 4 GB.

- Nearly all coefficients lie between minus 2 and plus 2, so a 2-bit integer part and a 13-bit fraction fit. Values outside minus 4 to 4 are clipped.

Q What else is in the served table?

A The key that says which feature each value belongs to. The paper notes a hash table stores a key beside each value, a string or a many-byte hash.

- Assume an 8-byte key (illustrative). The paper does not state its key size.

Q What is the whole-model saving?

A With keys: 16 bytes per coefficient (8 + 8) becomes 10 bytes (8 + 2). For 2 billion coefficients, 32 GB becomes 20 GB, a saving of **37.5%**, not 75%.

- A hash table also needs spare slots to stay fast. That multiplies both sizes by the same factor and leaves the percentage unchanged.

Q What does that do to the fleet?

A With 80 copies (20 data centers, 4 servers each, illustrative): 80 × 32 GB = 2.56 TB falls to 80 × 20 GB = 1.60 TB, 0.96 TB saved.

- The manager's plan would have been 0.64 TB, which is 0.96 TB short of the 1.60 TB the fleet needs.

Q Reframe it.

A A saving is true for the thing that was measured. Before applying a percentage to a bigger total, list what the total contains and which parts the percentage touches.

Basis: The 75% and its mechanism are from the paper (source-confirmed); the key size, the 2 billion coefficients and the 80 copies are illustrative; arithmetic recomputed.

PART 2

Features and data: hashing, rare features and subsampling

Choosing how to identify billions of features, deciding which to track, and shrinking the training stream without biasing it.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Feature hashing (the hashing trick)	Computing a feature's column as hash(value) modulo a fixed number of buckets, with no dictionary. The width is fixed, and two values can collide in one bucket, sharing one weight.
Keyed (non-hashed) table	A table that stores each coefficient under its full feature identifier, so distinct features never share a slot. It is still a hash table, but its keys are the full identifier or a long hash of it.
Poisson inclusion	Adding a new feature to the model only with probability p the first time it is seen. The number of sightings before inclusion is geometric with mean 1 divided by p.
Bloom filter inclusion	Using counting Bloom filters, compact structures that can report a false positive but never a false negative, to count the first n sightings of a feature before it enters the model.
Importance weight	A multiplier that makes a sampled example count for as many examples as it stands for. A kept example sampled with probability s gets weight 1 divided by s.
Unbiased estimate	An estimate whose average over the sampling process equals the true value. Importance weights make the subsampled loss unbiased in this sense.
Query-level sampling	Deciding to keep or drop all ads of one query together, because many features need shared work on the query phrase.
Signal and metadata index	A catalog of every input signal with its versions, status and consumers, used to find who is affected when a signal changes.
Where to read	McMahan et al.: Ad Click Prediction: a View from the Trenches (KDD 2013)

05 Hash the features, or keep them exact?

DESIGN CHOICE

SCENARIO

An engineer wants to cut memory by hashing every feature into a fixed number of buckets, as papers by Weinberger et al. and Chapelle did with 2 to the power of 24 buckets. The CTR paper tried it. Decide, using the paper and your own arithmetic.

HOW TO THINK ABOUT IT

Separate what the paper reports from what depends on numbers it does not publish. Then work out when each layout is smaller.

Q What does the paper report?

A It could not project down lower than several billion features without observable loss, saw no significant saving, and preferred interpretable non-hashed vectors (source-confirmed).

- Section 4.1 adds that hashing with collisions also did not help as a way to cut memory for rare features.

Q What is the difference between the two layouts?

A Hashed: a dense array of B buckets, with no keys, so B × 2 bytes at 16 bits per value. Keyed: N non-zero coefficients, each with an assumed 8-byte key and a 2-byte value, so N × 10 bytes.

- Hashing is smaller only when B is below 5 × N. Anything else is a collision cost with no memory benefit.

Q Which wins here?

A With N = 2 billion and B = 2 to the power of 32 (about 4.3 billion): hashed is 8.6 GB and keyed is 20 GB, so hashing looks smaller. With N = 1 billion the keyed table is 10 GB and hashing is no longer ahead.

- The paper publishes neither N, B nor the key size. You cannot rebuild its conclusion from its text, and it may not transfer to your data.

Q What else counts?

A Accuracy, which the paper says suffered, and interpretability: with collisions, one weight can no longer be traced to one real signal.

- Keyed does not mean collision-prone. With 8-byte keys, two billion features have an expected 0.1 colliding pairs. A 24-bit bucket space would collide constantly.

Q Reframe it.

A Do not copy a rejection or an adoption from a paper. Rebuild the comparison with your own sizes, and keep the paper's reasons, here accuracy and interpretability, as the extra terms.

Basis: The paper's finding and quotation are source-confirmed; N, B and the key size are illustrative; the break-even and collision arithmetic are recomputed.

06 Which rare features deserve a slot?

MEMORY

SCENARIO

In some of the paper's models, half the unique features occur only once in billions of examples. You cannot tell in advance which features will be rare, and a pre-pass that drops rare features costs an extra read and write of the data. Pick a method.

HOW TO THINK ABOUT IT

List what each method needs to remember, what it can get wrong, and what the paper's table says it costs.

Q Why not pre-filter the data?

A It needs another full read and write, and once rare features are dropped you cannot later try models that use them (source-confirmed).

- A feature seen once can never be useful, but you only know it was seen once after the data has passed.

Q What are the two online methods?

A Poisson inclusion adds a new feature with probability p, so the expected number of sightings before inclusion is 1 divided by p. Bloom filter inclusion counts the first n sightings with counting Bloom filters.

- A counting Bloom filter can report a feature as seen more often than it was (a false positive) but never less, so a feature occasionally enters early.

Q What does the paper's table say?

A Bloom n = 2 saves 66% of RAM for 0.008% AucLoss. Bloom n = 1: 55% for 0.003%. Poisson p = 0.03: 60% for 0.020%. Poisson p = 0.1: 40% for 0.006% (source-confirmed).

- Bloom filtering gives the better trade, as the paper says. Poisson p = 0.03 saves more than Bloom n = 1 but costs six times the accuracy.

Q Check the numbers against the half-are-singletons fact.

A Bloom n = 1 drops features seen once, and saves 55%. That fits half the unique features being singletons, plus the cost of keeping the rest.

- The percentages are relative to a baseline model whose size the paper does not give, so they cannot be turned into gigabytes.

Q Reframe it.

A When you cannot know which items matter until later, decide cheaply and online whether to remember them, and measure the accuracy price of being wrong.

Basis: Table 2 values, the singleton fact and the method descriptions are from the paper (source-confirmed); the consistency check is reasoned.

07 Subsample the non-clicked queries

DATA

SCENARIO

Training on every impression is expensive and clicks are rare. The paper keeps every query with at least one click and a fraction r of the rest, and weights the kept unclicked queries by 1 over r. You have 100 million queries a day with 3 ads each and a 2% click rate per ad.

HOW TO THINK ABOUT IT

Work at the level of the query, since that is the level of sampling. Count what is kept, then check that the weights give back the true totals.

Q How many queries have a click?

A The 2% is a rate per ad. With 3 ads and independent clicks (illustrative), the share of queries with at least one click is 1 − 0.98^3 = 5.88%.

- Using 2% for the share of queries would undercount clicked queries by about a factor of 3.

Q What is kept at r = 0.10?

A Clicked queries: 5,880,800. Unclicked: 94,119,200, of which 9,411,920 are kept. In all 15,292,720, or 15.3%, a reduction of **6.5 times**.

- Each kept unclicked query gets weight 10. Each clicked query keeps weight 1.

Q Do the weights restore the truth?

A Weighted impressions: 5,880,800 × 3 + 9,411,920 × 3 × 10 = 17,642,400 + 282,357,600 = 300,000,000, exactly the true count.

- The expected weighted loss equals the full-data loss, since a kept event with probability s has weight 1 over s (source-confirmed). Clicks stay at weight 1 and are all kept.

Q What goes wrong without weights?

A The model would see 6,000,000 clicks among 45,878,160 impressions, a CTR of 13.1% when the truth is 2%: predictions about 6.5 times too high.

- The paper says the loss and gradient are scaled by the weight, and that even fairly aggressive subsampling has a mild effect on accuracy.

Q Reframe it.

A Sampling that depends on the label must be paid for with a weight. Check any sampler by asking what the weighted totals come to, and compare them with the totals you started with.

Basis: The sampling rule, the weights and the mild-impact claim are from the paper (source-confirmed); 100 million queries, 3 ads, independent clicks and r = 0.10 are illustrative; all arithmetic recomputed.

08 Review the sampler

CODE REVIEW

SCENARIO

A colleague writes this sampler for the training stream and says it implements the paper's rule. The stream has one event per ad shown, with 3 ads per query and a 2% click rate per ad. The loss curve looks fine and the pipeline is fast. In an A/B test the model's predicted CTRs are far above the observed ones.

```
import random

def subsample(events, r=0.1):
    # events: one record per ad shown; fields query_id, clicked, features
    for e in events:
        if e.clicked or random.random() < r:
            yield e, 1.0          # (example, importance weight)
```

HOW TO THINK ABOUT IT

Compare the code line by line with the paper's rule. Then compute the click rate the model is trained on.

Q What does the code do?

A It decides for each event on its own: keep it if it was clicked, or with probability 0.1 if not, and give every kept event weight 1.

- The paper's rule decides for each query, and weights kept unclicked queries by 1 over r.

Q What is the first fault?

A No importance weight. With only per-event dropping, a clicked event is always kept and an unclicked one is kept 10% of the time.

- The training CTR becomes 0.02 ÷ (0.02 + 0.98 × 0.1) = 16.9%, against 2% in reality, so predictions run about 8.5 times too high.

Q What is the second fault?

A It samples ads one by one, not queries. A clicked query loses most of its unclicked ads, and the same query is processed again and again for other ads.

- The paper samples at the query level because many features need common processing of the query phrase, and so that the weight is right for every ad in the query.

Q State the fix.

A Decide once per query, from a hash of the query id so a replay gives the same result. Keep it if any ad was clicked, with weight 1, or with probability r, with weight 1 over r.

- Check by summing weights: they must equal the original event count, as in Question 7.

Q Reframe it.

A A pipeline can look healthy while its labels are biased. Add a check that compares weighted totals with unsampled totals, because loss curves cannot reveal this fault.

REFERENCE CODE

```
import zlib

def keep_query(query_id: str, r: float) -> bool:
    return (zlib.crc32(query_id.encode()) % 10_000) < int(r * 10_000)

def subsample(queries, r=0.1):
    # queries: one record per query, with a list of ad events
    for q in queries:
        if any(e.clicked for e in q.events):
            weight = 1.0
        elif keep_query(q.query_id, r):
            weight = 1.0 / r
        else:
            continue
        for e in q.events:
            yield e, weight
```

Basis: The rule is from the paper (source-confirmed); the 3 ads, 2% and r = 0.10 are illustrative; the 16.9% is arithmetic (recomputed); the fix was checked by hand and not run.

PART 3

The learning algorithm: FTRL-Proximal and per-coordinate rates

Why plain gradient descent is not enough, how to read the paper's comparison table, and what one update does to one coordinate.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Online learning	Updating the model right after each example arrives and then discarding the example. The paper stresses that it does not assume the examples are independent and identically distributed.
Logistic regression and LogLoss	A model that predicts p = sigmoid(w · x). After seeing label y it suffers the loss minus y log p minus (1 minus y) log (1 minus p). The gradient with respect to w is (p minus y) times x.
Online gradient descent (OGD)	The update w becomes w minus a learning rate times the gradient. It predicts well, but adding an L1 penalty to it almost never gives coefficients that are exactly zero.
L1 regularization and sparsity	A penalty on the sum of absolute weights. It can push weights to exactly zero, and a zero weight need not be stored or served.
FTRL-Proximal	An online algorithm that keeps a running quantity z per coordinate instead of the weight. A weight is exactly zero while the absolute value of z is at most the L1 strength. With no L1 term it matches OGD.
Per-coordinate learning rate	A separate rate for each feature: alpha divided by (beta plus the square root of the sum of that feature's squared gradients). A feature seen often settles, and a rare one stays quick to move.
Table-reading discipline	Before comparing rows of a results table, read the caption for the data each row was measured on. Rows measured on different datasets cannot be treated as one experiment.
Hand trace	Following one update through every formula with small numbers. It shows the mechanism and gives a reference to test an implementation against.
Where to read	McMahan et al.: Ad Click Prediction: a View from the Trenches (KDD 2013)

09 Why not plain gradient descent with an L1 penalty?

ALGORITHM CHOICE

SCENARIO

Serving needs a small model, so the team wants sparsity. They propose OGD with an L1 penalty. The paper's Table 1 compares FTRL-Proximal with RDA, FOBOS and OGD-Count. Decide, and say how much you trust the table.

HOW TO THINK ABOUT IT

Start with why sparsity matters for the served model. Then read the table together with its caption, not on its own.

Q Why does sparsity matter here?

A The number of non-zero coefficients determines the memory a stored model needs, and the model is replicated to many data centers, so the paper is more concerned with sparsity at serving than during training (source-confirmed).

- Question 4 gave 20 GB for 2 billion non-zero coefficients. Every zero removed shrinks every copy.

Q What goes wrong with OGD and L1?

A The paper says that adding a subgradient of the L1 penalty to the gradient will essentially never produce coefficients that are exactly zero.

- FOBOS and truncated gradient do induce sparsity, and RDA gives a better accuracy against sparsity trade than FOBOS. The paper has seen gradient-descent methods give better accuracy than RDA on its data.

Q What does FTRL-Proximal change?

A With no L1 term it produces exactly the same coefficients as OGD. It stores z instead of w, and a coefficient is exactly zero whenever the absolute value of z is at most the L1 strength.

- It keeps the accuracy of the gradient methods and gets the exact zeros.

Q How do you read Table 1?

A RDA: 3% more non-zero coefficients and a 0.6% AucLoss detriment. FOBOS: 38% more at 0.0%. OGD-Count: 216% more at 0.0% (source-confirmed).

- The caption says RDA and FOBOS were compared on a smaller prototyping set of millions of examples, and OGD-Count on a full-scale set. The rows are not one experiment.
- Applied to an assumed 20 GB model, 38% and 216% would mean about 27.6 GB and 63.2 GB at equal accuracy. That carries the percentages to a different model and data size, so it is a rough guide only.

Q Reframe it.

A Read a table with its caption. State which rows were measured on the same data before you draw a ranking from them, and say what you carried over when you reuse a percentage.

Basis: The algorithm properties, the OGD and L1 statement and Table 1 with its caption are from the paper (source-confirmed); the 20 GB model is illustrative; the 27.6 and 63.2 GB figures are arithmetic and an extrapolation.

10 Why does each coordinate get its own learning rate?

ALGORITHM

SCENARIO

The paper reports that per-coordinate learning rates cut AucLoss by 11.2% against a single global rate. A skeptic says that this must be a tuning artifact: both models just need a better learning rate. Explain the mechanism and answer the skeptic.

HOW TO THINK ABOUT IT

Use the paper's coin thought experiment with numbers, then ask what was held fixed in the comparison.

Q What is wrong with one global rate?

A A global schedule of 1 divided by the square root of t lowers the rate for every coordinate as rounds pass, whether or not that coordinate's feature was seen.

- In the paper's example one coin is flipped per round and each coin is its own logistic regression. A coin that is rarely flipped should still react quickly to new data.

Q What do the numbers look like?

A After 1,010 rounds a global rate is 1 ÷ sqrt(1,010) = 0.031 for every coin. A coin flipped 1,000 times deserves 0.032, and one flipped 10 times deserves 1 ÷ sqrt(10) = **0.316**.

- The rarely seen coin is being driven 10 times too slowly by the global schedule.

Q What is the paper's rate?

A For coordinate i: alpha ÷ (beta + the square root of the sum of squared gradients for i). Alpha is tuned under progressive validation, and beta = 1 is usually good enough because it stops early rates being too high (source-confirmed).

- A feature that has had many large gradients gets a small rate. One seen rarely keeps a large one.

Q Is the 11.2% a tuning artifact?

A The paper tested two identical models, one with a global rate and one with per-coordinate rates, and tuned the base parameter alpha separately for each (source-confirmed).

- So each had its own best alpha. It says AucLoss reductions of 1% are considered large in its setting, so 11.2% is a large win.

Q What does 11.2% mean in absolute terms?

A It is a relative reduction. If AUC were 0.80 (AucLoss 0.20, illustrative), a cut of 11.2% gives AucLoss 0.1776, AUC 0.8224.

- The paper warns that absolute metric values mislead and reports relative changes, so no absolute AUC is given.

Q Reframe it.

A When items arrive at very different rates, a shared clock treats them unfairly. Give each item a rate that depends on its own evidence, and check the comparison tuned both sides.

Basis: The mechanism, the formula, the 11.2% and the 1% remark are from the paper (source-confirmed); the 1,010 rounds, coin counts and the 0.80 AUC are illustrative arithmetic.

11 Trace one coordinate by hand

CALCULATION

SCENARIO

An interviewer gives you FTRL-Proximal with alpha = 0.1, beta = 1, L1 = 0.5 and L2 = 0. One feature has value 1. Its weight starts at 0, so the predicted probability is 0.5. The impression is clicked, twice in a row. Compute z, n and the weight after each click, and say what it shows.

HOW TO THINK ABOUT IT

Use the update from the paper's Algorithm 1, in order: weight, prediction, gradient, sigma, z, n.

Q What happens at the first click?

A Gradient g = (0.5 − 1) × 1 = −0.5. Sigma = (sqrt(0 + 0.25) − sqrt(0)) ÷ 0.1 = 5. Then z = 0 + (−0.5) − 5 × 0 = **−0.5**, and n = 0.25.

- Next weight: the absolute value of z, 0.5, is not above L1 = 0.5, so the weight is exactly 0.

Q What happens at the second click?

A The weight was 0, so the prediction is still 0.5 and g = −0.5. Sigma = (sqrt(0.5) − sqrt(0.25)) ÷ 0.1 = 2.071. Then z = −0.5 − 0.5 − 2.071 × 0 = −1.0, and n = 0.5.

- Weight now: the absolute value of z, 1.0, is above 0.5, so w = −(z − sign(z) × L1) ÷ ((beta + sqrt(n)) ÷ alpha + L2) = 0.5 ÷ ((1 + 0.7071) ÷ 0.1) = 0.5 ÷ 17.07 = **0.0293**.

Q What does that show about rare features?

A A feature seen once, with a click, is still exactly zero, so it costs nothing to serve. It needs enough evidence to push the absolute value of z past the L1 strength.

- That is the sparsity mechanism. The threshold is on accumulated evidence, not on a hand-picked count.

Q What would change with no L1 penalty?

A With L1 = 0, after the first click w = 0.5 ÷ ((1 + 0.5) ÷ 0.1) = 0.5 ÷ 15 = 0.0333, a non-zero weight from a single click.

- Every feature seen once would take a slot, which is the memory problem Question 6 tried to avoid.

Q Reframe it.

A Trace small cases by hand to see which quantity does what. The numbers also become a reference an implementation must reproduce exactly.

Basis: The update rule is the paper's Algorithm 1 (source-confirmed); the parameters and the clicks are illustrative; all arithmetic recomputed by hand.

12 Review the FTRL update

CODE REVIEW

SCENARIO

A colleague implements FTRL-Proximal. On a test with tiny data it produces the right signs, the sparsity looks right and the loss goes down. In a larger comparison against the reference implementation, the weights are always slightly different. Find out why.

```
def update(self, x, y):
    # self._weight(i) reads self.z[i] and self.n[i]
    p = self.predict(x)
    for i, v in x.items():
        g = (p - y) * v
        old_n = self.n.get(i, 0.0)
        self.n[i] = old_n + g * g
        sigma = (math.sqrt(self.n[i]) - math.sqrt(old_n)) / self.alpha
        self.z[i] = self.z.get(i, 0.0) + g - sigma * self._weight(i)
```

HOW TO THINK ABOUT IT

Compare the order of the operations with the paper's Algorithm 1, then trace the first few steps of Question 11 through the code.

Q Do the first two steps agree with the hand trace?

A Yes. At step 1 the weight is 0 and z = −0.5, n = 0.25. At step 2 the weight is still 0, so z = −1.0 and n = 0.5, as in Question 11.

- The error cannot show while the weight is 0, which is why a short test passes.

Q Where do they begin to differ?

A At step 3. The correct z is −1.5380 and the weight after it is 0.05575. The colleague's code gives z = −1.5342 and weight 0.05555.

- After step 4: 0.0802 against 0.0798. The gap is about 0.5% and the sign is right, so a coarse test cannot see it.

Q What is the fault?

A The code updates n first and then calls the weight function, which reads n. The term sigma × w in the z update must use the weight that made the prediction, w at the start of the round.

- Algorithm 1 computes w for each active feature first, predicts with those, and only then updates z and n.

Q Which test would have caught it?

A The paper's invariant: with L1 = 0 and L2 = 0, FTRL-Proximal produces exactly the same coefficients as OGD using the same per-coordinate rates. Compare the two over many steps.

- Also keep the hand trace of Question 11, with exact numbers, as a unit test.

Q State the fix.

A Compute each active weight once at the start of the round, predict with those, and use the same values in the z update.

- It also avoids computing every weight twice.

Q Reframe it.

A Order matters when an update reads state that it is about to change. Check an implementation against an invariant from the theory, and keep exact traces as tests.

REFERENCE CODE

```
def update(self, x, y):
    w = {i: self._weight(i) for i in x}          # weights played this round
    wx = sum(w[i] * v for i, v in x.items())
    p = 1.0 / (1.0 + math.exp(-wx))
    for i, v in x.items():
        g = (p - y) * v
        n = self.n.get(i, 0.0)
        sigma = (math.sqrt(n + g * g) - math.sqrt(n)) / self.alpha
        self.z[i] = self.z.get(i, 0.0) + g - sigma * w[i]
        self.n[i] = n + g * g
```

Basis: The algorithm and the equivalence with OGD at no L1 are from the paper (source-confirmed); the traces are arithmetic computed by hand (recomputed); the fix was checked by hand and not run on data.

13 Counts instead of squared gradients, and many variants at once

MEMORY

SCENARIO

Per-coordinate rates need the sum of squared gradients for each feature, a second number to store. The paper replaces it with two counts, and also trains many model variants together. Explain what is approximated, and where the memory really goes down.

HOW TO THINK ABOUT IT

Check the approximation on a small case, then count bytes per coordinate with and without sharing.

Q What is the approximation?

A Suppose every event with a feature has the same probability p = P ÷ (N + P), with P clicks and N non-clicks. Then the sum of squared gradients is P × N ÷ (N + P) (source-confirmed).

- The paper calls the assumption a terrible approximation in general, and says that for learning rates it works as well as the full sum.

Q Check it on numbers.

A P = 10, N = 990, so p = 0.01. The sum is 10 × 0.99^2 + 990 × 0.01^2 = 9.801 + 0.099 = 9.9. The formula gives 10 × 990 ÷ 1,000 = 9.9.

- It is exact when the model has learned p. The risk is in features whose probability varies between events.

Q Does it save memory by itself?

A Not by itself: two counts replace one number. It helps because the counts can be shared across model variants and stored with variable-length encodings, and most features need few bits (source-confirmed).

- The paper does not give byte figures for this, so treat the benefit as unquantified.

Q What does sharing across variants save?

A Assume an 8-byte key, a 2-byte value and 4 bytes of count state per coordinate (illustrative). Ten separate models cost 10 × 14 = 140 bytes per coordinate. Ten variants sharing key and counts cost 8 + 4 + 10 × 2 = 32 bytes, 4.4 times less.

- The paper's point: the incremental cost of a variant is only its extra coefficient values, and it saves network, CPU and disk too.

Q What about the single value structure?

A It stores one coefficient per feature shared by all variants that include it, with a bit-field of which variants use it. It saved an order of magnitude in RAM, with nearly identical relative performance (source-confirmed).

- It is lossy and ad hoc, so use it to screen many variants cheaply and then retrain the best candidates exactly.

Q Reframe it.

A A cheap approximation is worth using for the quantity that does not need to be exact. Be clear which part saves memory, here sharing, and which part only makes sharing possible.

Basis: The approximation, the sharing design and the order-of-magnitude result are from the paper (source-confirmed); the byte sizes and the count of 10 variants are illustrative; arithmetic recomputed.

PART 4

Serving, evaluation and operations

Replicating a continuously changing model, reading metrics with their noise, calibrating, scaling traffic and guarding the learner.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Snapshot and delta	A snapshot copies the whole model to a replica. A delta sends only the coefficients that changed since the last push.
Staleness	How long ago a replica's weights last matched the training state. A replica cut off from training keeps serving, with weights that grow older.
Progressive validation (online loss)	Scoring each example with the model's prediction made before it trains on that example, and aggregating those scores, here hourly. All data is used for both training and evaluation.
Standard error and margin	For a rate p measured on n events, the standard error is the square root of p(1 minus p) divided by n. A 95% margin is about 1.96 times that.
Isotonic regression	Fitting a correction curve that may only rise, by weighted least squares. Where neighboring points break the order, they are pooled to their average.
Power-law calibration	A correction of the form gamma times p to the power kappa, with the two constants fitted by Poisson regression on aggregated data.
Uncertainty score	The paper's bound on how far one training example could move the log-odds of a prediction, computed with one sparse dot product from the per-feature counts. Large for features seen rarely.
Feedback loop	The model's predictions decide which ads are shown, and what is shown decides what clicks are observed, so the data depends on the model.
Where to read	McMahan et al.: Ad Click Prediction: a View from the Trenches (KDD 2013)

14 Replicate a model that never stops changing

SERVING

SCENARIO

Training updates the weights after every example. The model must be copied to many data centers. The paper says only that trained models are replicated to many data centers for serving. Design how the copies are kept fresh, and what is stored where.

HOW TO THINK ABOUT IT

Separate what the paper states from what you must decide. Size a full copy and a delta before choosing.

Q What is in the paper, and what is not?

A Figure 1 shows streaming data going through probabilistic feature inclusion into parallelized training, then sparsification and calibration, and ending in model serving replicas, with progressive validation metrics drawn off training (source-confirmed).

- Nothing is said about how updates reach the replicas, how often, or in what format. That part is your design.

Q What does a serving replica hold?

A Only the non-zero weights. Training keeps z and count state for every tracked feature, including ones L1 will zero, so it is larger and lives in one place.

- With 2 billion non-zero weights at 10 bytes each, a replica is 20 GB, and 80 replicas hold 1.6 TB (illustrative).

Q What does a snapshot cost?

A 20 GB to each of 80 replicas is 1.6 TB per push. Pushed every 5 minutes, that is 1.6 TB ÷ 300 s = about 5.3 GB a second across the fleet.

- It is simple and easy to roll back, and it is the traffic you pay for on every push.

Q What does a delta cost?

A It equals the distinct coordinates touched in the interval. With 1.53 billion training events a day, 5 minutes is about 5.3 million events, times 300 non-zero values is up to 1.6 billion touches.

- That is close to the model's 2 billion coefficients, so everything depends on skew. If a few head features account for most touches, the distinct count is far smaller. Count distinct touched coordinates in the logs before choosing.

Q What if a replica loses its link to training?

A It keeps serving with older weights, because the model is a local copy. That is the availability benefit of replication.

- Alert on staleness beyond a threshold, and decide what happens past it, for example fall back to the last good snapshot.

Q Reframe it.

A Size both options in bytes per push before choosing. For a delta, the size depends on a number you have to measure, so measure it.

Basis: Figure 1's stages and the replication statement are from the paper (source-confirmed); the sizes, replica count and interval are illustrative; arithmetic recomputed; the push options are reasoned.

15 How good is a metric, and how noisy?

EVALUATION

SCENARIO

The paper evaluates with progressive validation, aggregated hourly, and also by slice (country, query topic, layout). A teammate says a 1% relative improvement in a slice is a clear win. Explain what progressive validation measures and how much noise a slice carries.

HOW TO THINK ABOUT IT

Define the metric's timing first. Then compute the standard error of a rate for an hour of all traffic and for a small slice.

Q What is progressive validation?

A Computing a gradient needs a prediction anyway, so that prediction, made before training on the example, is streamed out and aggregated hourly (source-confirmed).

- Each example is scored while it is still unseen by the model, which makes the score out of sample at that moment. All data serves for both training and evaluation.

Q What are its limits?

A An hour's number averages the model over the versions it passed through during that hour. Two models can be compared only when scored on exactly the same data.

- The paper says it compares only metrics computed from the same data, and reports relative changes against a baseline, because relative changes are far more stable over time.

Q How noisy is the whole stream?

A 10 billion events a day (illustrative) is 417 million an hour. At a 2% rate the standard error is sqrt(0.02 × 0.98 ÷ 417,000,000) = 0.0000069, 0.034% of the rate. The 95% margin is about **0.07%**.

- On the whole stream, a 1% relative change is large against its noise.

Q How noisy is a small slice?

A A slice with 0.1% of traffic has 417,000 events an hour. The standard error is 0.000217, 1.1% of the rate, and the 95% margin is about **2.1%**.

- A 1% change in that slice, in one hour, is inside the noise. Use a longer window, and compare models on the same events.

Q Why does the paper add a visualization?

A A small aggregate gain on one metric may hide a mix of gains and losses in different countries or topics. GridViz shows each metric across hundreds of slicings against a control, with column width for impression count (source-confirmed).

- Wide columns are the ones to trust, and narrow ones need more data. The tool lets you scan for outliers before shipping.

Q Reframe it.

A Attach a margin to every number you read, and scale the margin with the size of the group it came from. Small groups need more time or more data.

Basis: Progressive validation, hourly aggregation, relative reporting and GridViz are from the paper (source-confirmed); the 10 billion events, 2% and the 0.1% slice are illustrative; the standard errors are recomputed. The margins are for a rate, not for AUC.

16 Calibrate, and say how sure the model is

EVALUATION

SCENARIO

The auction needs honest probabilities, and it also benefits from knowing how sure the model is about each one. The paper adds a calibration layer and proposes an uncertainty score. Explain both, and what limits them.

HOW TO THINK ABOUT IT

Define calibration on a small table of buckets. Fix it two ways, then ask what each method cannot promise.

Q What does calibrated mean?

A If you pool the impressions where the model said p, about a fraction p of them should be clicked. Bias is the gap between the average predicted and the average observed CTR on a slice (source-confirmed).

- Example buckets (illustrative): predicted 0.02 and observed 0.020; predicted 0.05 and observed 0.046; predicted 0.10 and observed 0.044; predicted 0.20 and observed 0.150. The model over-predicts at the high end.

Q What are the two fixes?

A A power law, gamma times p to the power kappa, fitted by Poisson regression. Or isotonic regression: a piecewise correction that may only rise, fitted by weighted least squares.

- The paper says the piecewise approach significantly reduced bias at both the high and low ends compared with the power law.

Q How does isotonic regression treat noisy buckets?

A If observed rates fall where they should rise, the points are pooled to their average. With equal weights, the observed values 0.020, 0.046, 0.044 and 0.150 become 0.020, 0.045, 0.045 and 0.150.

- The curve is monotone, so ranking is kept, and noise between neighbors is smoothed.

Q What cannot calibration promise?

A The paper says that without strong extra assumptions, the feedback loop in the system makes it impossible to give theoretical guarantees for the impact of calibration. It also separates concerns: the auction can be tuned apart from the learner.

- Check calibration by slice. A correction that fits the average can be wrong for one market.

Q What does the uncertainty score do?

A Standard confidence intervals do not fit: the models are online, heavily regularized, not assumed independent, and a standard method inverts an n by n matrix with n in the billions. The score is an upper bound on how far one example can move the log-odds, from one sparse dot product with the per-feature counts (source-confirmed).

- It is large for rarely seen features. It was checked on synthetic labels drawn from a ground-truth model, where errors correlated with the score and matched a bootstrap of 32 models. The paper shows this as a plot, not as numbers.

Q Reframe it.

A Correct the size of the number, and report how much to trust it. Know the limits of both: a correction is checked slice by slice, and a confidence heuristic is validated on cases where the truth is known.

Basis: Calibration, both fixes, the feedback-loop limit and the uncertainty score are from the paper (source-confirmed); the bucket table is illustrative; the pooled values are arithmetic (recomputed).

17 Ten times the traffic

SCALE

SCENARIO

Imagine 100 billion predictions a day. A manager asks what breaks first, and whether to add machines or change the design.

HOW TO THINK ABOUT IT

Scale each quantity by ten, then sort the results by whether they follow traffic or follow the number of features.

Q What happens to rates?

A 115,741 becomes 1,157,407 a second on average, and 347,222 becomes 3,472,222 at peak. The count of table lookups at peak is 1.04 billion a second.

- Lookups are still spread over many cores and servers. Add machines.

Q What happens to the training stream?

A With the Question 7 subsampling, 15.3% of events are kept, so 15.3 billion training events a day, 177,000 a second.

- It scales with traffic. The online update touches about 300 coordinates per event.

Q What happens to the model?

A Its size depends on how many distinct features earn a slot, not on traffic. The paper gives no growth rate, so it is a number to measure.

- Probabilistic inclusion keeps singletons out as traffic grows. The more traffic, the more features pass the threshold.

Q What happens to replication?

A Memory per replica barely changes, but the number of replicas grows with serving capacity, so total memory grows. With a model of 20 GB, 800 replicas would hold 16 TB.

- A snapshot push also grows with the replica count, which favors deltas.

Q What happens to the measurements?

A The standard error falls with the square root of the count, about 3.2 times, so noise is no longer the limit.

- Slices of 0.01% of traffic are as readable as 0.1% was before.

Q Reframe it.

A Sort the quantities by what they scale with. Some follow traffic and yield to machines. Others follow the number of features or of replicas, and need a design decision.

Basis: The qualitative scale is from the paper (source-confirmed); the 100 billion, the 15.3% kept and 800 replicas are illustrative; all arithmetic recomputed.

18 Fake clicks and a broken signal

RISK

SCENARIO

An online learner trusts the clicks it sees. A security reviewer asks what happens if a ring generates fake clicks, and an engineer asks what happens if an input signal silently changes meaning. The paper discusses neither fraud nor attacks.

HOW TO THINK ABOUT IT

Estimate how far a burst of fake clicks moves a rate, then ask which part of the system catches each failure.

Q How far can fake clicks move a rate?

A An ad has 10 million impressions at 2%, 200,000 clicks. A ring adds 1,000,000 fake clicks: 1,200,000 ÷ 10,000,000 = **12%**, six times the true rate.

- The online learner updates the features of that ad as the clicks arrive, so its probability rises within the day.

Q Where would you defend?

A Before the join: filter invalid clicks, since the learner has no way to tell a fake click from a real one. Watch each ad's rate against what the model expected, with the uncertainty score as context.

- The paper does not cover this, so it is reasoning. A spike far above expectation, on an ad with little history, deserves review before it is trained on.

Q What does a silently changed signal do?

A A signal that starts returning a constant, or a new unit, shifts every prediction that uses it, and the model keeps training on the changed values.

- The paper's metadata index annotates signals with versions and deprecation, and can find every consumer of a changed signal (source-confirmed).

Q How would you notice?

A Sliced metrics against a control, as GridViz does, show a pattern in the affected slices before the aggregate moves. Alert on calibration by slice as well.

- Keep the ability to roll the model back, which is easier with snapshots than with a continuous stream.

Q Reframe it.

A A learner that adapts quickly to real behavior adapts quickly to fake behavior too. Put the guard where the data enters, and keep a way to go back.

Basis: The signal index is from the paper (source-confirmed); the paper does not cover fraud, so the defenses are reasoned and untested; the ad figures are illustrative arithmetic.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #2. The one source is Google's KDD 2013 paper, whose full text was read on 2026-10-17. The paper reports scale only qualitatively (billions of events a day, billions of coefficients), so every concrete size in this book is an illustrative assumption. It says nothing about how model updates reach the serving replicas, how clicks are joined to impressions, or click fraud, so those answers are design arguments made for this book and were not run on real systems.

- McMahan, Holt, Sculley, Young, Ebner, Grady, Nie, Phillips, Davydov, Golovin, Chikkerur, Liu, Wattenberg, Hrafnkelsson, Boulos, Kubica: Ad Click Prediction: a View from the Trenches, KDD 2013, full text read 2026-10-17
https://research.google.com/pubs/archive/41159.pdf